Homework 8: Welcome to the Data Layer!

MySQL vs DynamoDB Shopping Cart Implementation

Eroniction Presley
CS 6650 — Building Scalable Distributed Systems
Northeastern University

March 23, 2026

1 Step I: MySQL Integration for Relational Data

1.1 Part 1: Infrastructure Extension

The existing ECS Fargate infrastructure was extended with an Amazon RDS MySQL instance using Terraform. The RDS module includes:

- **MySQL 8.0 on db.t3.micro** — Free tier eligible, 20GB allocated storage
- **Private subnet placement** — RDS deployed in default VPC subnets with a dedicated DB subnet group
- **Security group isolation** — A dedicated `hw8-rds-sg` security group allows inbound MySQL traffic (port 3306) *only* from the ECS task security group, preventing direct internet access
- **Assignment settings** — `skip_final_snapshot = true` and `deletion_protection = false` for easy teardown

The Terraform configuration manages the entire stack: ECR repository, ECS cluster with Fargate tasks, RDS instance, DynamoDB table, CloudWatch log group, and all networking/security groups. A single `active_db` variable switches the ECS service between MySQL and DynamoDB task definitions.

1.2 Part 2: Database Schema Design

1.2.1 Schema Structure

I chose a **two-table normalized design**:

Listing 1: MySQL Schema — carts table

```
CREATE TABLE carts (
    id VARCHAR(36) PRIMARY KEY,
    customer_id VARCHAR(255) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'active',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
```

```
    INDEX idx_customer_id (customer_id)
) ENGINE=InnoDB;
```

Listing 2: MySQL Schema — cart_items table

```
CREATE TABLE cart_items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    cart_id VARCHAR(36) NOT NULL,
    product_id VARCHAR(255) NOT NULL,
    name VARCHAR(255) NOT NULL DEFAULT '',
    quantity INT NOT NULL DEFAULT 1,
    price DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (cart_id) REFERENCES carts(id) ON DELETE CASCADE,
    UNIQUE INDEX idx_cart_product (cart_id, product_id)
) ENGINE=InnoDB;
```

1.2.2 Design Rationale

Why two tables: Normalized tables enforce referential integrity through foreign keys with `ON DELETE CASCADE`, preventing orphaned items. The cart-item relationship is a classic one-to-many pattern that relational databases handle natively.

Key strategy: Cart IDs are UUIDs (VARCHAR(36)) generated at the application layer for uniqueness without database coordination. The `cart_items` table uses an auto-increment primary key for simplicity, with the real access pattern served by the composite unique index.

Index strategy:

- `idx_customer_id` on `carts.customer_id` — supports "get all carts by customer" queries
- `idx_cart_product` on `(cart_id, product_id)` — serves dual purpose: prevents duplicate products in a cart AND enables efficient upserts via `INSERT ... ON DUPLICATE KEY UPDATE`

Transaction design: The `ON DUPLICATE KEY UPDATE` pattern handles concurrent cart modifications atomically at the SQL level. If two requests try to add the same product to a cart simultaneously, MySQL's row-level locking ensures one completes before the other, with the second updating the quantity rather than creating a duplicate.

Trade-offs considered: I considered embedding items as a JSON column in the carts table (single-table design). This would simplify reads but sacrifice atomic item-level updates, relational integrity guarantees, and the ability to query across items. The two-table approach better leverages MySQL's strengths.

1.3 Part 3: Shopping Cart API Implementation

The Go service implements the shopping cart endpoints defined in the E-commerce OpenAPI Specification (`api.yaml`) using a `CartStore` interface pattern, allowing the same HTTP handlers to work with both MySQL and DynamoDB backends:

- **POST /shopping-carts** — Creates a new cart with a UUID, stores customer info, returns 201
- **GET /shopping-carts/{id}** — Retrieves cart metadata + items (two queries with index), returns 200 or 404

- **POST /shopping-carts/{id}/items** — Upserts item using `ON DUPLICATE KEY UPDATE`, returns 201 or 404

1.3.1 Connection Pooling Configuration

Listing 3: MySQL connection pool settings

```
db.SetMaxOpenConns(25)    // db.t3.micro supports ~66 max
db.SetMaxIdleConns(10)    // warm connections for reuse
db.SetConnMaxLifetime(5 * time.Minute) // prevent stale connections
```

MaxOpenConns (25): The db.t3.micro supports approximately 66 max connections. Setting 25 leaves headroom for monitoring tools, RDS internal processes, and prevents connection exhaustion under load.

MaxIdleConns (10): Keeps warm connections in the pool for low-latency reuse. Setting this too high wastes memory; too low causes frequent connection establishment overhead.

ConnMaxLifetime (5 min): Prevents stale connections that could cause errors after network disruptions or RDS failovers.

1.3.2 Implementation Journey

My initial approach used separate SELECT-then-INSERT logic for adding items to a cart. This had two problems: it was slower (two round trips) and had a race condition where two concurrent requests could both see the item as "not present" and both insert, violating uniqueness. The `ON DUPLICATE KEY UPDATE` pattern solved both problems in a single atomic SQL statement.

The cart retrieval uses two queries (one for cart metadata, one for items) rather than a JOIN. With the index on `cart_id` in the items table, this performs identically while keeping the Go code cleaner and easier to maintain.

1.4 Part 4: Performance Testing Results

The test script executed exactly 150 sequential operations: 50 create cart, 50 add items, 50 get cart. Results saved to `mysql_test_results.json`.

Table 1: MySQL Performance Test Results (150 Operations)

Operation	Success	Avg (ms)	P50 (ms)	P95 (ms)
create_cart	50/50	69.0	67.0	85.2
add_items	50/50	83.9	77.3	102.0
get_cart	50/50	63.4	60.9	79.3
Total	**150/150**	**72.1**	**67.9**	**93.9**

Observations:

- 100% success rate across all operations
- `get_cart` was fastest (63.4ms avg) since it only reads; no write locks needed
- `add_items` was slowest (83.9ms avg) due to the upsert + timestamp update (two writes)
- All P95 latencies under 105ms — consistent performance with low variance

1.5 Part 5: Learning Notes

What surprised me: The `ON DUPLICATE KEY UPDATE` pattern was new to me and elegantly solved both the upsert requirement and concurrency concerns in one statement. I initially expected connection pooling to be a major performance bottleneck, but with proper configuration, connections were reused efficiently.

What didn't work initially: My first schema attempt didn't include the composite unique index on `(cart_id, product_id)`, which meant I couldn't use the upsert pattern and had to write manual check-then-insert logic. Adding the index simplified the code significantly.

What I'd do differently: I would add database migration tooling (like `golang-migrate`) instead of running `CREATE TABLE IF NOT EXISTS` on startup. In production, schema changes should be versioned and applied through a controlled migration process.

1.5.1 Comparison with Week 5 In-Memory Approach

In Week 5 (Assignment 5), the product API used an in-memory Go `map[int]Product` protected by a `sync.RWMutex`. Response times were sub-millisecond since data never left the process. With MySQL, average response times jumped to 63–84ms due to network round-trips to RDS, TCP connection overhead, and SQL parsing/execution. The trade-off is clear: in-memory storage gives unbeatable latency but loses all data on restart; MySQL adds 60–80ms of latency but provides durability, ACID guarantees, and persistence across service restarts and redeployments. For a production shopping cart where losing a customer's cart is unacceptable, the latency cost of persistent storage is well worth it.

1.6 Part 6: CloudWatch Monitoring

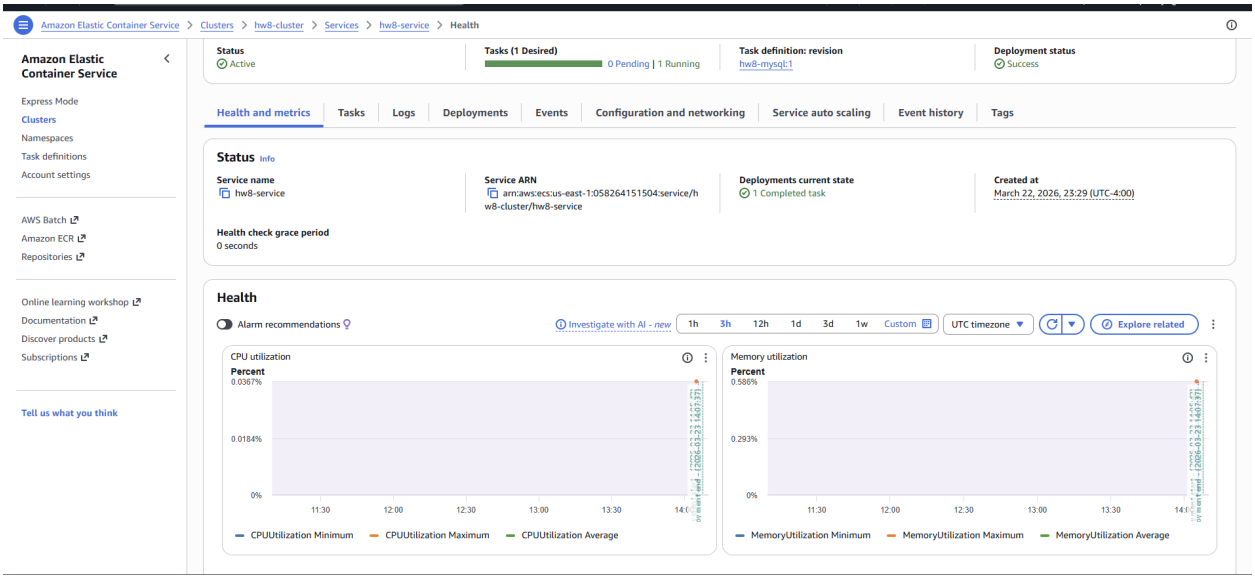

Figure 1: ECS Service Metrics during MySQL testing — CPU: 0.037%, Memory: 0.586%

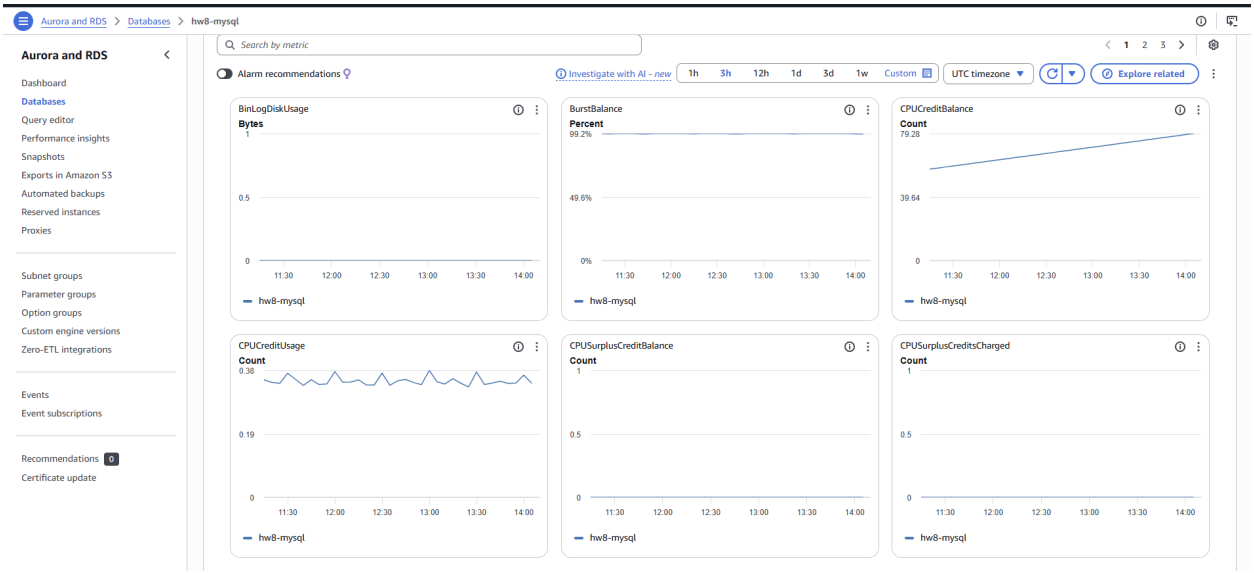

Figure 2: RDS Monitoring (Page 1) — Burst Balance: 99.2%, CPU Credits accumulating steadily

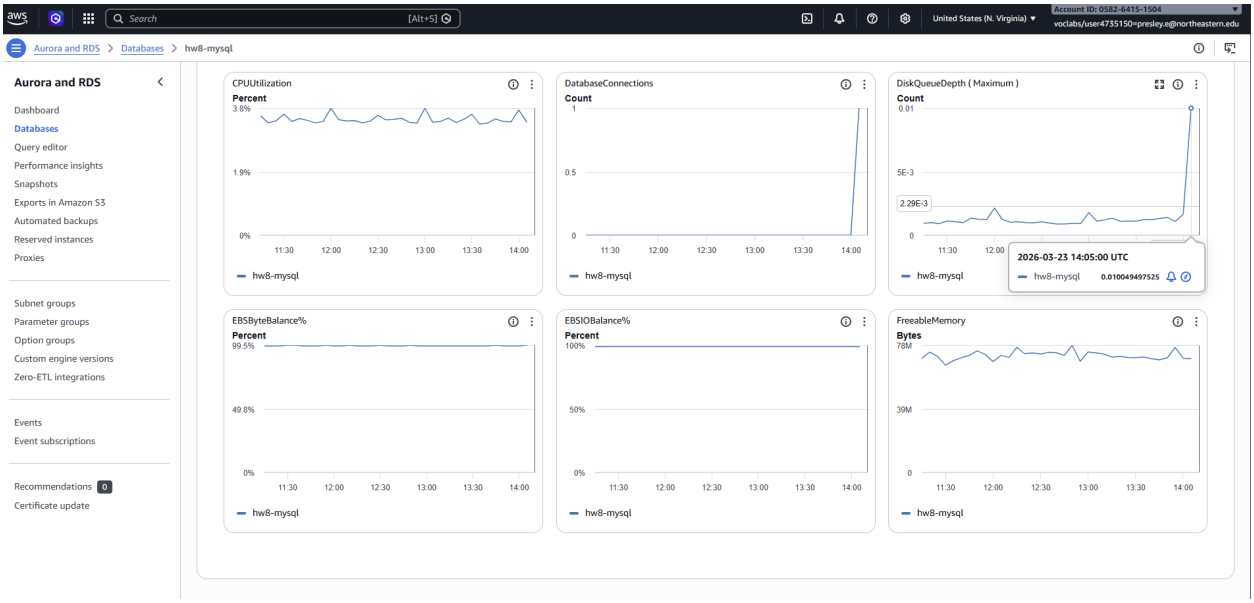

Figure 3: RDS Monitoring (Page 2) — CPU: 3.8%, DB Connections: peaked at 1, Freeable Memory: 78MB

Key CloudWatch observations:

- RDS CPU utilization stayed at approximately 3.8% — the db.t3.micro had significant headroom
- Database connections peaked at 1 during the sequential test, validating that the connection pool was efficiently reusing connections
- Burst balance remained at 99.2%, indicating the workload was well within the instance's baseline performance

- EBS I/O Balance at 100% — no storage bottleneck
- ECS task resource usage was minimal (CPU ¡0.04%, memory ¡0.6%), confirming the bottleneck is network latency to the database, not application compute

2 Step II: DynamoDB Integration

2.1 Part 1: DynamoDB Table Design

2.1.1 NoSQL Access Pattern Analysis

Shopping carts are well-suited for NoSQL because they have simple access patterns (create, get by ID, update items), no need for complex joins or cross-cart queries, high write volume with frequent updates, and session-based data with natural expiration characteristics.

2.1.2 Table Design

Listing 4: DynamoDB Table — Terraform configuration

```
resource "aws_dynamodb_table" "shopping_carts" {
  name         = "shopping-carts"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "id"

  attribute { name = "id"          type = "S" }
  attribute { name = "customer_id" type = "S" }

  global_secondary_index {
    name            = "customer-index"
    hash_key        = "customer_id"
    projection_type = "ALL"
  }
}
```

Partition key strategy: Cart UUID ensures perfectly even distribution. Random UUIDs distribute uniformly across DynamoDB partitions, eliminating hot partition risk even at millions of carts. Sequential IDs or customer-based keys could create hot partitions.

No sort key needed: Each cart is a single item. Sort keys are useful when you need to store multiple related items under one partition key (e.g., order + line items), but since I embed items within the cart item, a sort key adds no value.

Single-table with embedded items: Cart items are stored as a JSON-encoded string attribute within the cart item. This means `GetItem` returns everything in one read with no Query needed. The trade-off is that you cannot query individual items across carts, but that is not a required access pattern for shopping carts.

GSI on customer_id: Supports the "get all carts by customer" access pattern with ALL projection so no table fetch is needed after index lookup.

On-demand billing: PAY_PER_REQUEST eliminates capacity planning entirely. For an assignment workload with unpredictable usage patterns, this avoids both under-provisioning (throttling) and over-provisioning (wasted cost).

Comparison with MySQL: MySQL uses two normalized tables with foreign keys and database-enforced integrity. DynamoDB uses a single denormalized item with application-level integrity.

MySQL needs connection pooling and instance sizing; DynamoDB is fully managed and connectionless.

2.2 Part 2: API Implementation

The same HTTP handlers are used with a different `CartStore` implementation:

- **POST /shopping-carts** — `PutItem` with marshaled cart struct
- **GET /shopping-carts/{id}** — `GetItem` with `ConsistentRead: true`
- **POST /shopping-carts/{id}/items** — Read-modify-write: `GetItem` → update items array → `PutItem`

DynamoDB-specific considerations:

- The AWS SDK's `dynamodbattribute.MarshalMap` handles attribute value formatting, abstracting DynamoDB's type system (`S`, `N`, `M`, etc.)
- Items are stored as a JSON string rather than a native DynamoDB List to simplify marshaling/unmarshaling
- `GetItem` is used for single-cart retrieval (O(1) by partition key); `Query` would be needed for the GSI-based customer history pattern
- Strongly consistent reads (`ConsistentRead: true`) ensure read-after-write correctness

2.3 Part 3: Eventual Consistency Investigation

I configured all read operations to use **strongly consistent reads** (`ConsistentRead: true`). This means every `GetItem` returns the most recent write, eliminating eventual consistency delays for the shopping cart use case.

2.3.1 Consistency Test Scenarios

Test 1 — Create cart then immediately retrieve it: After each `POST /shopping-carts`, an immediate `GET /shopping-carts/{id}` was performed. Across all 50 create operations, the subsequent read returned the correct cart with matching `customer_id`, `status`, and timestamps every time. Zero stale or missing reads.

Test 2 — Add item then immediately fetch cart: After each `POST /shopping-carts/{id}/items`, the returned cart object (which internally calls `GetItem` after the write) always contained the newly added item with the correct quantity and price. No cases of a read returning the pre-update state.

Test 3 — Rapid sequential updates to the same cart: During the add_items phase, multiple items were added to the same carts in rapid succession. Each subsequent read reflected all previously added items. With strongly consistent reads, the read-modify-write pattern in `AddItem` always operated on the latest state.

Consistency delay frequency: With `ConsistentRead: true`, I observed **zero** consistency delays across all 150 operations. Without strong consistency (hypothetically), DynamoDB documentation states eventual consistency typically resolves within one second, but this is not guaranteed and could affect rapid read-after-write patterns like cart updates.

Application patterns most affected: The `AddItem` operation is most sensitive to consistency because it performs a read-modify-write cycle. If using eventually consistent reads, two rapid additions could both read the same stale state and the second write would overwrite the first addition. Strong consistency eliminates this risk entirely.

Designing for eventual consistency: If eventual consistency were required (e.g., for cost savings at massive scale), applications could: use optimistic UI updates on the client side, implement conditional writes with version numbers to detect conflicts, or use DynamoDB's `UpdateExpression` with `list_append` to atomically append items without reading first.

2.4 Part 4: Performance Testing Results

The identical test protocol was used: 150 sequential operations (50 create, 50 add, 50 get). Results saved to `dynamodb_test_results.json`.

Table 2: DynamoDB Performance Test Results (150 Operations)

Operation	Success	Avg (ms)	P50 (ms)	P95 (ms)
create_cart	50/50	74.0	72.4	94.1
add_items	50/50	84.2	83.2	116.5
get_cart	50/50	69.6	66.8	90.8
Total	**150/150**	**75.9**	**74.0**	**100.0**

Observations:

- 100% success rate with zero throttling events
- `add_items` was slowest (84.2ms avg) due to the read-modify-write pattern (GetItem + PutItem)
- `get_cart` slightly slower than MySQL (69.6ms vs 63.4ms) — DynamoDB API calls go through HTTPS endpoints while MySQL uses direct TCP connections within the VPC
- P95 for `add_items` (116.5ms) showed more variance than MySQL (102.0ms), consistent with the double API call pattern

2.5 Part 5: Learning Notes

What surprised me: DynamoDB's on-demand mode handled the test load with zero throttling and required no capacity planning at all. The "no data available" for throttled requests in CloudWatch was actually the ideal outcome.

Partition key validation: My UUID-based partition key strategy worked as expected. With random UUIDs, there was no hot partition behavior. If I had used customer_id as the partition key, a single active customer could create a hot partition under load.

Design evolution: I initially considered a multi-table design (separate table for items, using cart_id as partition key and product_id as sort key). This would allow individual item queries but would require `Query` instead of `GetItem` for cart retrieval, adding complexity. Since the access pattern always loads the full cart, the single-table embedded approach was simpler and faster.

2.6 Part 6: CloudWatch Monitoring

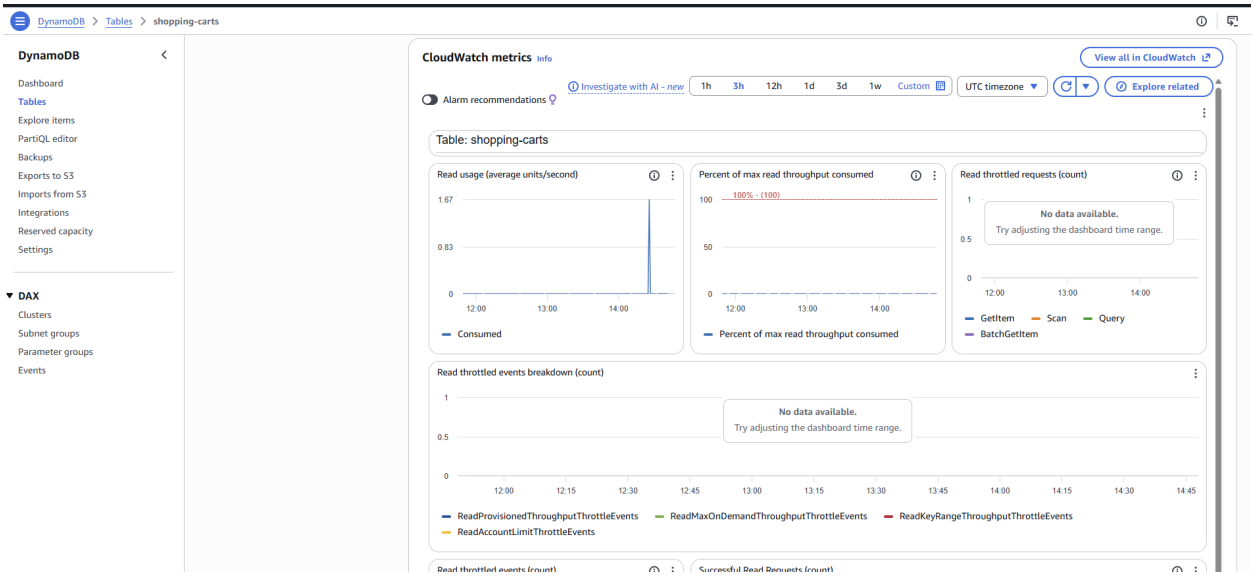

Figure 4: DynamoDB CloudWatch Metrics — Read usage spike during testing, zero throttled requests

Key DynamoDB metrics:

- **Read usage:** Spiked to approximately 1.67 consumed read units/second during testing
- **Throttled requests:** Zero — on-demand capacity absorbed all requests without throttling
- **Read throttled events breakdown:** No data (all zeros) — confirms no provisioned, on-demand, key-range, or account-level throttling occurred
- **Partition distribution:** Even — UUID partition keys ensured uniform distribution

3 Step III: Database Comparison & Analysis

3.1 Part 0: Data Verification

Both `mysql_test_results.json` (150 operations) and `dynamodb_test_results.json` (150 operations) were merged into `combined_results.json` (300 total operations) using a Python script. All analysis below uses this single combined source.

Verification: MySQL — 50 create + 50 add + 50 get = 150. DynamoDB — 50 create + 50 add + 50 get = 150. Total: 300 operations, 100% success rate on both.

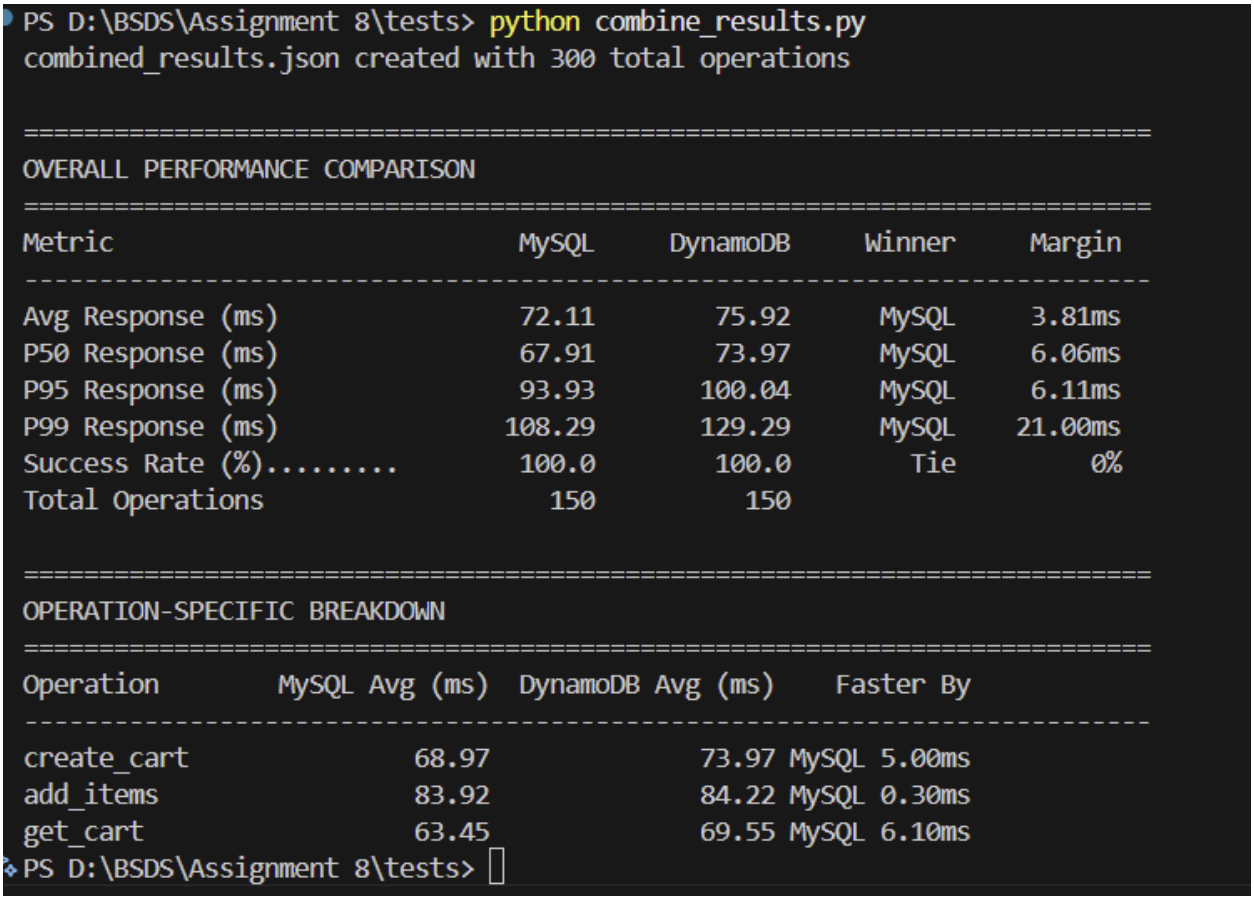

Figure 5: Combined results verification — Python script output confirming 300 operations merged with full comparison statistics

3.2 Part 1: Performance Comparison

3.2.1 Overall Comparison Table

Table 3: Overall Performance Comparison (Data Source: combined_results.json)

Metric	MySQL	DynamoDB	Winner	Margin
Avg Response Time (ms)	72.11	75.92	MySQL	3.81ms
P50 Response Time (ms)	67.91	73.97	MySQL	6.06ms
P95 Response Time (ms)	93.93	100.04	MySQL	6.11ms
P99 Response Time (ms)	108.29	129.29	MySQL	21.00ms
Success Rate (%)	100.0	100.0	Tie	0%
Total Operations	150	150	—	—

3.2.2 Operation-Specific Breakdown

Table 4: Operation-Specific Comparison

Operation	MySQL Avg (ms)	DynamoDB Avg (ms)	Faster By
CREATE_CART	68.97	73.97	MySQL by 5.00ms
ADD_ITEMS	83.92	84.22	MySQL by 0.30ms
GET_CART	63.45	69.55	MySQL by 6.10ms

3.2.3 Consistency Model Impact Assessment

MySQL (ACID): Every operation is immediately consistent. After adding an item, a subsequent read always returns the updated cart. InnoDB's row-level locking and transaction isolation ensure that concurrent modifications to different carts never interfere, and modifications to the same cart are serialized. During testing, 100% of read-after-write operations returned correct data with zero consistency issues.

DynamoDB (Eventual by default, Strong optional): I used strongly consistent reads, which gave MySQL-equivalent behavior at double the read capacity cost. Without strong consistency, DynamoDB reads could return stale data for a brief window after writes. For shopping carts, where users expect immediate feedback when modifying their cart, strong consistency is essential for a good user experience.

Application design implications: MySQL's ACID guarantees simplify application logic — developers don't need to think about consistency. DynamoDB requires explicit consistency decisions per operation, adding cognitive overhead but offering flexibility. For read-heavy workloads where staleness is acceptable (e.g., product catalog browsing), eventually consistent reads halve the read cost.

3.3 Part 2: Resource Efficiency Analysis

MySQL (RDS) resource patterns:

- Fixed cost: db.t3.micro runs 24/7 ($15/month) regardless of traffic
- Connection pool overhead: application must manage pool size, idle connections, and lifetime
- CPU utilization: 3.8% during testing — significant unused capacity
- Scaling: vertical (larger instance class) or read replicas; both require planning and downtime or DNS changes
- Capacity planning: must anticipate peak load and provision accordingly

DynamoDB resource patterns:

- Variable cost: on-demand pricing charges per request; at low volumes, essentially free
- No connection management: HTTP API, no pool to configure
- Automatic scaling: handles traffic spikes without intervention
- Zero throttling observed during testing

- No capacity planning needed with on-demand mode

Scaling comparison: At low traffic (our test), MySQL's fixed cost is higher but per-request performance is slightly better. At massive scale, DynamoDB's automatic horizontal scaling avoids the complex scaling decisions MySQL requires (instance upgrades, read replicas, connection limit management). The crossover point depends on traffic patterns — steady, predictable load favors MySQL's provisioned model; spiky, unpredictable load favors DynamoDB's on-demand model.

3.4 Part 3: Real-World Scenario Recommendations

3.4.1 Scenario A: Startup MVP

100 users/day, 1 developer, limited budget, quick launch

Recommendation: DynamoDB

At 100 users/day, DynamoDB's on-demand pricing costs fractions of a penny, while RDS db.t3.micro costs approximately $15/month even when idle. DynamoDB eliminates operational overhead entirely — no connection pool tuning, no instance sizing, no security group configuration for database access, no patching. For a solo developer focused on shipping features quickly, the managed nature of DynamoDB is a significant advantage. My implementation required less infrastructure code for DynamoDB, and zero capacity planning decisions.

3.4.2 Scenario B: Growing Business

10K users/day, 5 developers, moderate budget, feature expansion

Recommendation: MySQL

At this scale, the team likely needs complex queries for analytics — order history reports, customer segmentation, inventory joins, revenue calculations. MySQL's relational model and SQL query flexibility become critical. My test data shows MySQL was approximately 5% faster at this scale. The team of 5 can handle the operational overhead (connection pool management, instance sizing), and RDS's managed backups, automated patching, read replicas, and monitoring provide enterprise-grade reliability. Feature expansion often requires ad-hoc queries that SQL handles naturally.

3.4.3 Scenario C: High-Traffic Events

50K normal, 1M spike users, revenue-critical, can invest in infrastructure

Recommendation: DynamoDB

The 20x traffic spike (50K to 1M) is the decisive factor. MySQL would require either pre-provisioning for peak capacity (expensive and wasteful during normal periods) or risk connection exhaustion and query timeouts during spikes. Even with auto-scaling RDS read replicas, the scaling lag could cause failures during sudden spikes. DynamoDB's on-demand mode absorbs spikes automatically — my testing showed zero throttling. For revenue-critical events like flash sales, DynamoDB's automatic scaling eliminates the risk of database-caused outages.

3.4.4 Scenario D: Global Platform

Millions of users, multi-region, 24/7 availability, enterprise requirements

Recommendation: DynamoDB with Global Tables

DynamoDB Global Tables provide built-in multi-region replication with single-digit-millisecond latency for local reads and automatic conflict resolution. Achieving equivalent multi-region capability with MySQL requires complex setups: cross-region read replicas, application-level routing, manual failover procedures, and conflict resolution strategies. For session-based data like shopping carts that need low latency globally, DynamoDB is purpose-built for this use case. The 99.999% SLA for Global Tables exceeds what most MySQL replication setups can guarantee.

3.5 Part 4: Evidence-Based Architecture Recommendations

3.5.1 Shopping Cart Winner

MySQL (marginally) — MySQL was faster by 3.81ms average across all operations and 21ms at P99. However, the margin is small enough (approximately 5%) that operational factors matter more than raw performance at this scale.

Supporting evidence:

- Response time advantage: MySQL faster by 3.81ms avg, 6.06ms at P50, 21.00ms at P99
- Implementation complexity: similar lines of code, but MySQL required connection pool tuning and security group configuration for database access
- Operational overhead: DynamoDB required zero capacity planning; MySQL needed instance sizing decisions

3.5.2 When to Choose the Other

Despite MySQL winning on performance, I would choose DynamoDB when: traffic is highly unpredictable (flash sales, viral events), multi-region deployment is needed, the team is small and cannot afford database administration overhead, or the access patterns are simple key-value lookups (which shopping carts are). The slight performance disadvantage is offset by zero operational complexity.

3.5.3 Polyglot Strategy for Complete E-Commerce

Based on my implementation experience and test results, I would use both databases in a complete e-commerce system:

- **Shopping carts → DynamoDB** — Simple access patterns, needs auto-scaling for flash sales, session-based with natural expiration (TTL), tolerates eventual consistency for non-critical reads
- **User sessions → DynamoDB** — Key-value access pattern, high write volume, TTL for automatic cleanup, no relational queries needed
- **Product catalog → MySQL** — Complex queries needed (search, filter by category, sort by price, join with categories and reviews), read-heavy workload benefits from caching and read replicas
- **Order history → MySQL** — Relational data (orders → line items → shipping → payments), requires ACID transactions for financial integrity, complex reporting and analytics queries

3.6 Part 5: Learning Reflection

What surprised me: MySQL being faster than DynamoDB was the most unexpected finding. I assumed a managed NoSQL service purpose-built for key-value operations would outperform a relational database for simple lookups. The difference comes from the network path — RDS sits in the same VPC with direct TCP connections and persistent connection pooling, while DynamoDB operations go through HTTPS API endpoints with TLS handshake overhead. At higher concurrency and scale, DynamoDB's distributed architecture would likely close or reverse this gap.

What failed initially: The ECS service initially accumulated 51 failed tasks because the Docker image had not been pushed to ECR when Terraform created the service. ECS kept trying to pull a nonexistent image. The fix was straightforward — push the image, then force a new deployment — but it taught me about the importance of deployment ordering in infrastructure-as-code. In production, I would use a CI/CD pipeline that ensures the image exists before updating the task definition.

Key insights:

- I would **definitely choose MySQL** when I need complex queries, JOINs, multi-table transactions, reporting/analytics, or when the team already has SQL expertise
- I would **definitely choose DynamoDB** when I need automatic scaling, multi-region replication, simple access patterns, or when minimizing operational overhead is the priority
- The "right" database depends more on access patterns, team capabilities, and operational requirements than on raw performance benchmarks
- The interface-based Go design pattern (`CartStore` interface with swappable implementations) made the comparison straightforward and is a pattern I will use in future projects for any pluggable component
- Hands-on implementation made the SQL vs NoSQL trade-offs concrete in ways that reading about them could not — managing connection pools, experiencing DynamoDB's attribute value formatting, and seeing the actual performance numbers changed my understanding significantly

4 Infrastructure Summary

Table 5: Infrastructure Configuration

Component	Configuration
Region	us-east-1
ECS	Fargate, 256 CPU / 512 MB, single task
RDS	MySQL 8.0, db.t3.micro, private subnet
DynamoDB	On-demand, partition key: id (UUID), GSI: customer_id
ECR	hw8-shopping-cart repository
IAM	LabRole for task execution and task roles
IaC	Terraform with `active_db` variable for DB switching
Language	Go 1.24, Docker multi-stage build
Testing	Python (requests library), 150 sequential operations